ASTROWORLD

Frontend / UI / UX Build Prompts

Incremental prompt sequence for an AI coding agent

Mobile-first social platform for amateur astronomers

Stack: Expo SDK 54 · TypeScript · Expo Router · NativeWind · Zustand · TanStack Query · Reanimated · react-native-maps · Sentry

Phase: frontend on mock data, no backend · Designed for later Figma swap-in

Feed one prompt at a time · review · commit · advance

How to use this document

Build the frontend on **mock data only** — no Supabase, no PowerSync, no auth. This lets you validate the whole app visually before wiring real data. Feed the prompts to your agent **one at a time**, open the app and check the result after each, and commit to git before advancing. Each prompt assumes the previous one succeeded. Two screens (the celestial starmap renderer and the mock-to-real data swap) are intentionally excluded — they each need their own dedicated sequence.

Standing rules (paste before Prompt 1)

These also belong in your `.cursorrules` / `CLAUDE.md` so they load every turn. Restated here so the prompt sequence is self-contained.

Stack & workflow

- Stack is fixed: Expo SDK 54, TypeScript, Expo Router, TanStack Query, Zustand, NativeWind, Reanimated, react-native-maps, Sentry. Do not substitute or add libraries without asking.
- After every change, run `npx tsc --noEmit` and the linter. Do not move on until both are clean.
- All data this phase is mock data from local files. No network, no backend.
- Build only what each prompt asks. Stop and summarize when done. Keep diffs small.

Theme (Aurora) — use tokens, never hardcode

- bg `#000000` · surface `#23313e` · teal `#86cdc3` · lavender `#bb90e3` · blue `#52a5dc` · text `#ecf3f2` · muted `#bac7d2`

Replaceability (so Figma designs drop in later)

- Screens in `/app` own data, state, navigation, behavior — never raw styling or hardcoded visual values.
- All visual rendering lives in `/components/ui` presentational components: typed props in, no logic, no data fetching, no store access.
- Every UI import comes from the barrel `@/components/ui`. Keep exported names & prop shapes stable — that barrel is the Figma swap point.

The prompt sequence

PROMPT 1

Project scaffold

Initialize a new Expo app (SDK 54) with TypeScript and Expo Router using the latest `create-expo-app`. Set up the project structure with an `/app` directory for routes, plus `/components`, `/lib`, `/mocks`, `/store`, and `/theme` folders. Install and configure NativeWind for Tailwind styling, and verify a basic styled screen renders. Do not add other libraries yet. Confirm the app boots on Expo Go and `tsc` passes.

PROMPT 2

Theme and design tokens

Create a theme system in `/theme`. Define the Aurora color tokens and configure NativeWind's `tailwind.config.js` so they're usable as utility classes (`bg-surface`, `text-primary`, `text-muted`, `accent-teal`, etc.). Set default background to pure black and default text to `#ecf3f2`. Create the presentational primitives in `/components/ui`, each in its own file with an exported typed props interface: `ThemedText`, `Card`, `Button` (teal primary / lavender secondary), `Screen` (layout wrapper), `Input`, `Avatar`, `Tag`. These are the ONLY components allowed to contain styling. Every value must reference a theme token — no hardcoded literals. Show them on a temporary preview screen.

PROMPT 2.5

Component barrel (the swap seam)

Create `/components/ui/index.ts` that re-exports every presentational component and its props type from a single entry point. All screens must import UI components from this barrel only (e.g. `import { Card, Button } from '@/components/ui'`), never from individual files. This barrel is the seam where Figma-based implementations will later be swapped in: as long as exported names and prop types stay stable, components behind it can be replaced without touching any screen.

PROMPT 3

Navigation shell

Build the bottom tab navigation with Expo Router: five tabs — Home, Starmap, Log (center, emphasized), Community, Profile. Use placeholder screens that show the tab name. Style the tab bar for dark theme with teal active icons and muted inactive icons. No hamburger menu. Confirm navigation between all five tabs works.

PROMPT 4

Mock data layer

Create realistic mock data in `/mocks` as typed TypeScript files for: User (profile + equipment list), ObservationSession (date, location, observed objects, equipment used, notes), Post (author, body, optional image, reaction/comment counts), MarketplaceListing (title, price, condition, category, seller, photos), and TonightSky (moon phase, conditions, top objects). Define a shared `/lib/types.ts` and make the mocks conform. Create synchronous accessor functions (e.g. `getMockSessions()`) that screens use as stand-ins for real data. *These accessors are the seams where real data plugs in later.*

PROMPT 5

Local state with Zustand

Set up Zustand stores in `/store` : `useSessionDraftStore` (in-progress observation log), `useUIStore` (red-light mode toggle, active filters), `useProfileStore` (current user from mocks). Keep stores small and typed. Add a red-light-mode boolean to the UI store but don't wire the visual effect yet. Confirm state updates with a temporary test button.

PROMPT 6

Home / Tonight's Sky screen

Build the Home screen (Journey 1 entry point). Top: a "Tonight's Sky" card (location, % clear, moon phase, top object) from mock TonightSky data, with "Plan session" and "Relocate" buttons. Below: a stats row (sessions, following, objects) and a "From your region" feed preview using mock Posts. Compose `/components/ui` primitives only — no raw styling in the screen. Calm, uncluttered, no infinite scroll. Buttons may navigate or no-op for now.

PROMPT 7

Log an observation flow

Build the Log flow (Journey 2) as a multi-step screen. Step 1: conditions auto-populated from mock data (editable). Step 2: search a mock celestial-object list and select observed objects. Step 3: pick equipment from the mock user's gear. Step 4: optional note + "Share to community" toggle. On submit, write to `useSessionDraftStore` , then clear and show a success state. No network — works fully offline by design. Screen holds logic only; all visuals via `/components/ui` .

PROMPT 8

Community feed and post detail

Build the Community tab (Journey 3). A scrollable feed of mock Posts (author avatar, name, gear used, body, optional image placeholder, reaction/comment counts). A search/filter bar that filters the feed by equipment model. Tapping a post opens Post Detail (full post + mock replies + a reply input writing to local state). Tapping an author opens a basic Profile view. All visuals via presentational components.

PROMPT 9

Profile and equipment

Build the Profile screen (Journey 4 end state, Journey 3 destination). Show the mock user's name, approximate location, stats, equipment list, and a gallery grid of session photos (placeholders). Add an "Add equipment" flow that searches a mock equipment database and adds a selection to the profile store. Clearly distinguish viewing your own profile (editable) vs. another user's (follow button).

PROMPT 10

Marketplace

Build the Marketplace tab (Journey 5). A grid/list of mock Listings with filters for category, price range, and distance. Tapping a listing opens a detail screen (photos, specs, condition, price, seller summary with session count and rating). Add a "Message seller" button opening a simple mock message thread, and a "Save listing" toggle in local state. Browsing works without auth.

PROMPT 11

Onboarding

Build the first-launch onboarding (Journey 4): three value-framing screens (plan / log / connect), a mock "Sign up with Apple" button, a location-permission priming screen (explain why before the OS prompt), and an "Add your first telescope" step. On completion, set an onboarded flag in the profile store and route to Home. Mock auth state only — no real authentication.

PROMPT 12

Animations and red-light mode

Add Reanimated for restrained, "calm not addictive" motion: smooth screen transitions, a gentle press animation on the Button primitive, a subtle fade-in for feed items. Wrap animations *inside* the presentational components (press animation in `Button`, fade in a `FeedItem`), not in screens. Implement red-light mode now: when the UI store's boolean is on, swap to a red-only palette (dark red background, red text/accents, no blues/greens/whites). Add a toggle in Profile or settings.

PROMPT 13

Error tracking (Sentry)

Integrate Sentry for React Native following the current Expo setup guide. Configure it to capture crashes and errors. Add a temporary dev-only "trigger test error" button to confirm events arrive, then make it dev-only or remove it. Never log user PII.

PROMPT 14

Maps — relocation sites

Integrate react-native-maps for "My Sites" / relocation (Journey 1). Show a map with mock saved stargazing sites as markers, each with a name and mock light-pollution rating. Tapping a marker shows site details. Allow dropping/saving a new site (local state). This is the terrestrial map only — the celestial starmap renderer is a separate, later task.

Deliberately excluded

STARMAP RENDERER

The celestial map and AR mode is the hardest, most specialized frontend piece — rendering star positions from ephemeris data, device-orientation AR, performance on older phones. It needs its own focused prompt sequence and probably a spike. Do not let the agent attempt it as a one-line task; tackle it deliberately after the shell works, the way you're treating offline sync.

MOCK → REAL DATA SWAP

Wiring Supabase / PowerSync / FastAPI is a separate phase. Because screens read through the mock accessor functions from Prompt 4, that swap happens at those seams — screens stay untouched. This gets its own prompt sequence once the mock-data app looks and feels right.

Running this well

- **One prompt per focused session** where practical. Long sessions accumulate context and cost; fresh sessions reload the rules file but drop the bloat.
- **Commit after each working prompt.** Checkpoints let you start fresh fearlessly and `git reset` instead of arguing the agent back to a good state.
- **Review incrementally.** Small accepted diffs keep context lean and catch drift early.
- **Drift check:** the fastest review is "is there any hardcoded color or style in an `/app` file?" If yes, pull it back into `/components/ui` — that single check protects the whole Figma-swap property.